

Git : Initiation et bonnes pratiques

1. Installation de Git

Site officiel et téléchargement

- [Site officiel de Git](#)
- Télécharger la dernière version pour Windows :
 - [Git for Windows \(x64\)](#)
 - Ou directement depuis [GitHub](#) (version 2.55.0 au 20/08/2026)

Installation

- Exécuter le fichier `.exe` téléchargé et suivre les instructions.
- Après installation, utiliser **Git Bash** (recommandé) ou tout autre terminal (PowerShell, CMD, etc.).

Après installation, utilisez : Git Bash ;

- Vérifier l'installation : `git --version`

2. Configuration initiale

Chaque utilisateur doit configurer son identité Git.

```
# Configurer son identité (remplacer par vos infos)
# Configurer son nom
git config --global user.name "Prénom Nom"

# Configurer son adresse e-mail
git config --global user.email "adresse@carnus.fr"

# Définir le nom de la branche créée par défaut
git config --global init.defaultBranch main
```

- Ces 2 premières commandes indiquent l'auteur des commits.
- `init.defaultBranch main` indique que les nouveaux dépôts utiliseront main comme branche initiale.
- **Vérifier la configuration :**

```
git config --global --list
```

On peut également vérifier une valeur particulière :

```
git config --global user.name
git config --global user.email
```

```
# Voir l'aide
git config --help
git config -h      # Aide rapide dans le terminal
git help config    # Ouvre l'aide dans le navigateur (Documentation plus
complète)
```

Palier 1 : init, add, commit, log, status

3. Initialisation d'un dépôt Git

```
# Créer ou utiliser un dossier de travail :
cd Exp_Git

# Initialiser le dépôt Git
git init # Crée un dossier caché `.git`

# Vérifier l'état du dépôt (suivi des fichiers)
git status # Aucun fichier n'est encore suivi par Git
```

- Cela crée un **répertoire .git caché** : à ne pas modifier ou supprimer, c'est lui qui gère le dépôt Git.

4. Ajout et suivi des fichiers

Créer par exemple un fichier :

```
index.html
```

Puis vérifier l'état :

```
git status
```

```
# Ajouter le fichier à la zone de staging (index)
git add index.html

# Vérifier l'état
```

```
git status # Le fichier apparaît maintenant comme étant prêt à être
commité.

# Retirer un fichier de la zone de staging (retire le fichier du suivi Git
sans supprimer le fichier du disque)
git restore --staged index.html

# Vérifier l'état
git status
```

Git permet de préparer précisément ce que l'on souhaite enregistrer dans un commit.

Le cycle fondamental est :

```
Répertoire de travail
    |
    | git add
    v
Zone de staging
    |
    | git commit
    v
Dépôt Git
```

5. Ignorer des fichiers avec **.gitignore**

Certains fichiers ne doivent généralement pas être versionnés :

- fichiers temporaires ;
- fichiers générés automatiquement ;
- fichiers de compilation ;
- journaux (logs) ;
- fichiers contenant des informations locales ;
- fichiers propres à certains logiciels.

Créer un fichier **.gitignore** à la racine du projet.

Ajouter les règles suivantes pour ignorer tous les fichiers **.txt** :

```
# Ignorer tous les fichiers .txt
*.txt
```

- Vérifier que les fichiers ignorés n'apparaissent plus :

```
git status
```

- Pour afficher également les fichiers ignorés :

```
git status --ignored
```

Important : `.gitignore` empêche normalement un fichier non suivi d'être proposé par Git. Il ne supprime pas le suivi d'un fichier déjà commité.

Pour retirer un fichier déjà suivi tout en le conservant sur le disque :

```
git rm --cached fichier.txt
```

6. Créer un commit

```
# Ajouter tous les fichiers
git add . # ou git add -A ou git add --all

# Créer le commit
git commit -m "premier commit - ajout de tous les fichiers dans le dépôt"

# Vérifier l'état
git status
```

7. Consulter l'historique des commits

```
# Voir l'historique
git log

# Voir l'historique en une ligne
git log --oneline

# Afficher l'historique sous forme de graphe :
git log --oneline --graph --decorate --all

# Afficher les modifications associées aux commits :
git log -p

# Afficher les informations d'un commit :
git show <commit>
```

Modifier le dernier commit

Si vous avez oublié une modification ou souhaitez corriger le message du dernier commit :

```
# Modifier le dernier commit
git commit --amend -m "Nouveau message"
```

Exemple :

```
git add logo.png
git commit --amend -m "Ajouter le nouveau logo"
```

`--amend` modifie le dernier commit. Il faut donc l'utiliser avec prudence lorsque le commit a déjà été partagé avec d'autres personnes.

Palier 2 : suppression, renommage, diff, restauration (apprendre à se tromper et corriger)

8. Modifier et comparer des fichiers

Modifier un fichier (ex: index.html)

```
# Vérifier l'état
git status # Le fichier modifié doit apparaître

# Voir les différences
git diff

# Ajouter le fichier modifié
git add index.html

# Voir les modifications actuellement dans le staging :
git diff --staged

# (Optionnel) : Si vous ne souhaitez finalement pas inclure le fichier dans
le prochain commit :
git restore --staged index.html

# Créer le commit :
git commit -m "Modifier la page d'accueil"
```

9. Suppression et renommage de fichiers

```
# Supprimer un fichier
git rm style.css

# Annuler la suppression
git restore style.css

# Renommer un fichier
git mv logo.png logoY.png

# Commit
git commit -m "Modification du nom du fichier logo"
```

Restaurer un fichier depuis un ancien commit

Pour récupérer la version d'un fichier telle qu'elle existait dans un ancien commit :

```
git restore --source=<commit> -- <fichier>
```

Exemple :

```
git restore --source=abc1234 -- index.html
```

Cette commande restaure le contenu du fichier à partir du commit indiqué dans le répertoire de travail.

Notes :

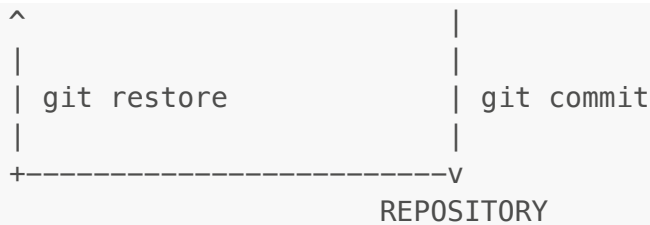
```
git restore <fichier>
# restaurer depuis l'état de référence actuel.
```

```
git restore --staged <fichier>
# retirer du staging.
```

```
git restore --source=<commit> -- <fichier>
# récupérer la version du fichier provenant d'un autre commit.
```

Le cycle de base Git

```
                                git add
WORKDIR -----> STAGING
```



Palier 3 : branches et fusions simples (collaborer)

10. Gestion des branches

Créer et utiliser une branche

```

# Lister les branches :
git branch

# Créer une nouvelle branche
git branch ChangeLogo

# Lister les branches
git branch
# Changer de branche
git switch ChangeLogo
# Ou créer et utiliser immédiatement la branche :
git switch -c ChangeLogo

# Modifier le fichier html (changer le logo)
# Commit
git add .
git commit -m "Modifier le logo"

# Revenir sur main
git switch main

# Fusionner la branche
git merge ChangeLogo

# Supprimer la branche
git branch -d ChangeLogo

```

Créer une branche et travailler dessus

```

# Créer et switcher vers une nouvelle branche
git switch -c ChangeTexte

# Modifier le texte dans le fichier html
# Commit
git add .

```

```
git commit -m "Modifier le texte"

# Revenir sur main
git switch main

# Modifier le fichier html
# Commit
git add .
git commit -m "Modifier le texte sur main"

# Fusionner (conflit possible)
git merge ChangeTexte

# Un conflit peut apparaître si les mêmes lignes ont été modifiées sur les
deux branches.
```

Résoudre un conflit

Lorsqu'un conflit apparaît, Git peut insérer des marqueurs dans le fichier :

```
<<<<<< HEAD
Version présente sur main
=====
Version présente sur ChangeTexte
>>>>>> ChangeTexte
```

Il faut :

- ouvrir le fichier ;
- choisir ou combiner les bonnes modifications ;
- supprimer les marqueurs de conflit ;
- enregistrer le fichier ;
- vérifier l'état

```
git status
```

Ajouter le fichier résolu :

```
git add index.html
```

Terminer la fusion :

```
git commit -m "Résoudre le conflit sur le titre"
```


Abandonner une fusion : Si vous souhaitez annuler complètement la fusion en cours :

```
git merge --abort
```

Branches et fusion (merge)

```
main:    (o)--(o)--(o)-----M---->
           \          /
feature:  (o)----- (o)
```

- créer une branche (**feature**) depuis **main**.
- travailler, commit, puis **git merge feature** dans **main** (point 'M').

Résolution de conflit

```
main:    (o)----(X)
           \
feature:  (Y)
           \
           M
```

X et Y ont modifié une même partie du fichier.

Lors du merge, Git ne peut pas déterminer automatiquement quelle version conserver.

Il demande donc une intervention humaine.

Palier 4 rebase, reset, stash

11. Rebase interactif (avancé)

```
# Lancer un rebase
git rebase -i HEAD~3
# Ouvre les trois derniers commits pour permettre de les réorganiser,
fusionner ou modifier.
```

12. Restaurer un fichier depuis un commit

```
git restore --source=<LeNuméroDuCommit> -- <fichier>
```

13. Résumé des commandes utiles

```
git --version    # Afficher la version de Git
git init        # Initialiser un dépôt
git clone <url>  # Cloner un dépôt distant
git status      # Voir l'état du dépôt
git add <fichier>    # Ajouter un fichier au staging
git add .       # Ajouter les modifications du répertoire courant au
staging
git commit -m "message"    # Créer un commit
git log         # Afficher l'historique
git log --oneline    # Afficher l'historique en version condensée
git log --oneline --graph --decorate --all # Afficher l'historique sous
forme de graphe
git diff        # Voir les modifications non staged
git diff --staged    # Voir les modifications staged
git restore <fichier>    # Abandonner les modifications locales du fichier
git restore --staged <fichier>    # Retirer un fichier du staging
git restore --source=<commit> -- <fichier>    # Restaurer un fichier depuis
un commit
git rm <fichier>    # Supprimer un fichier et l'indiquer à Git
git mv <ancien> <nouveau>    # Renommer/déplacer un fichier
git branch      # Lister les branches
git switch <branche>    # Changer de branche
git switch -c <branche>    # Créer et utiliser une branche
git merge <branche>    # Fusionner une branche
git merge --abort    # Annuler une fusion en cours
git branch -d <branche>    # Supprimer une branche fusionnée
git revert <commit>    # Créer un commit annulant un ancien commit
git reset --soft <commit>    # Déplacer HEAD en conservant le staging
git stash       # Mettre temporairement les modifications de côté
git stash pop   # Récupérer les modifications stashées
git pull       # Récupérer et intégrer les modifications distantes
git push       # Envoyer les commits vers le dépôt distant
git remote -v  # Afficher les dépôts distants
```

TP : Git en ligne de commande

Objectifs

- Maîtriser les commandes de base de Git.
- Savoir créer, modifier, commiter, brancher et fusionner.
- Comprendre la résolution de conflits.

Prérequis

- Git installé et configuré.
- Un terminal (Git Bash, PowerShell, etc.).
- Un éditeur de texte (VS Code, Notepad++, etc.).

Énoncé

1. Initialisation et premier commit

1. Créez un dossier `tp1_git` sur le Bureau.
2. Placez-vous dans ce dossier `cd tp1_git`, initialisez un dépôt Git `git init`.
3. Configurez votre nom et email si ce n'est pas déjà fait.
4. Vérifiez : `git status`
5. Créez un fichier `index.html` avec le contenu suivant :

```
<!DOCTYPE html>
<html>
<head>
  <title>Mon premier site</title>
</head>
<body>
  <h1>Bienvenue sur mon site</h1>
</body>
</html>
```

6. Ajoutez ce fichier à la zone de staging et faites un commit avec le message "Ajout de la structure de base".

2. Modification et historique

1. Modifiez le fichier `index.html` en ajoutant un paragraphe :

```
<p>Ce site est versionné avec Git.</p>
```

2. Vérifiez les modifications avec `git diff`.
3. Ajoutez le fichier modifié à la zone de staging et faites un commit avec le message "Ajout d'un paragraphe".
4. Affichez l'historique des commits avec `git log --oneline`.
5. Créez un bug volontaire, puis utilisez :

```
git restore --source=commit_id -- fichier
```

→ Restaure le fichier dans l'état où il se trouvait dans le commit indiqué, sans déplacer la branche.

3. Création et fusion de branches

1. Créez une branche `ajout-style`.
 - `git switch -c ajout-style`
2. Basculez sur cette branche.

3. Créez un fichier `style.css` avec le contenu suivant :

```
body {  
  font-family: Arial;  
  background-color: lightblue;  
}
```

4. Ajoutez et committez ce fichier avec le message "Ajout du fichier CSS".

5. Revenez sur la branche `main`.

6. Fusionnez la branche `ajout-style` dans `main`.

7. Supprimez la branche `ajout-style`.

8. Visualisez l'historique en graphe :

```
git log --oneline --graph
```

4. Résolution de conflit

1. Créez une branche `modif-accueil`.

2. Basculez sur cette branche et modifiez le `<h1>` dans `index.html` :

```
<h1>Bienvenue sur mon site version 2</h1>
```

3. Committez avec le message "Modification du titre".

4. Revenez sur `main` et modifiez le même `<h1>` :

```
<h1>Site en construction</h1>
```

5. Committez avec le message "Changement du titre".

6. Fusionnez `modif-accueil` dans `main`. Un conflit devrait apparaître.

7. Résolvez le conflit en gardant le texte "Bienvenue sur mon site version 2".

8. Committez la résolution

5. Ignorer des fichiers

1. Créez un fichier `notes.txt` avec du texte quelconque.

2. Configurez `.gitignore` pour ignorer tous les fichiers `.txt`.

3. Vérifiez avec `git status` que `notes.txt` n'est pas suivi. Ou `git status --ignored`

6. Annulation et rebase

1. Créez un fichier `script.js` et committez-le avec le message "Ajout d'un script".

2. Annulez ce commit avec `git reset --soft HEAD~1`.

3. Vérifiez que le fichier est toujours présent et que ses modifications sont toujours dans la zone de staging.
4. Recommitez-le avec le message "Ajout d'un script (v2)". (`git commit -m "Ajout d'un script (v2)"`)

Solution attendue

- Un dépôt Git avec un historique propre, sans conflits non résolus.
- Les fichiers `index.html`, `style.css`, `script.js` et `.gitignore` correctement versionnés.